AI Image Generator

3. System Architecture

Version 2.3
23 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Images, video and music on one GPU: Z-Image Turbo, SDXL fine-tunes, Qwen-Image, FLUX.1, LoRAs, LTX-Video, Wan 2.1, MusicGen and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and an Admin panel that manages models, LoRAs and downloads and shows live system statistics.

3. System Architecture

3.1 Component overview

The application is a single Python process. Gradio serves the browser interface and calls plain Python functions; those functions use sixteen project modules, one resident image model, at most one resident video or music model, and the upscaler.

Component	File	Responsibility
Web application	app.py	Builds the Studio and History tabs, owns the ModelManager and the Upscaler, orchestrates one generation end to end, renders the details report, mounts the Admin panel.
Admin panel	admin_ui.py	Eight Admin sub-tabs and their handlers: models, LoRAs, add, downloads, system, settings, logs.
Model registry	models.py	Built-in specs, local checkpoints and admin-added models merged with registry overrides; ModelManager (load, swap, free VRAM, lock), samplers, family-aware LoRA activation, compel prompt encoding, quantised loaders for Z-Image, Qwen-Image and FLUX.
Persistent registry	registry.py	models/registry.json: settings, custom models and overrides, LoRA metadata (family, trigger words, weight, source, enabled).
Downloads	downloads.py	Source inspection (Hugging Face repo or file, Civitai, URL, local path), the background JobManager with progress, resumable file downloads, repo snapshots with byte progress, deletion of model files.
Theme and shared UI	ui_theme.py, ui_common.py	Colours, Gradio theme and CSS; dropdown choices, header, model cards, tiles, key-value tables and the downloads list.
Log buffer	applog.py	Tees stdout and stderr into a ring buffer for the Logs tab.
Video and music engine	media.py	Profiles for LTX-Video, Wan 2.1, MusicGen and Stable Audio; MediaManager (load straight to GPU, bf16 text-encoder cache, unload, generate_video, generate_music); prepare_for() GPU coordination with the image ModelManager.
Video and Music tabs	media_ui.py	Handlers and layout of both tabs: generation, frame-wise upscaling, soundtrack muxing, model defaults, chips, examples, media details report.
Video and audio files	video_io.py	ffmpeg (from imageio-ffmpeg) wrappers: streamed MP4 encoding, frame reading, first-frame thumbnails, audio muxing; WAV writing and waveform pictures.
Prompt parser	prompt_parser.py	Turns free text into a Settings object: model, LoRAs, size, steps, guidance, count, seed, style, upscale target. Pure function, no GPU.
Upscaler	upscaler.py	Wraps Real-ESRGAN through spandrel; tiled x4 passes; resizes or crops to the requested target.
GPU telemetry	gpu_info.py	Static GPU facts from torch; live temperature, power, clocks, fan and load from nvidia-smi.
System telemetry	sysinfo.py	CPU, memory, disks, network, GPU extras, power, software and process facts for the Admin panel, built on psutil and nvidia-smi.
Database	db.py	SQLite schema, migration, insert, update, fetch, search, per-row get, aggregate stats, command-line viewer.
CLI generator	generate.py	Standalone batch generation with any model and LoRA, without the web UI.

3.2 Generation pipeline

For every request the sequence is:

1 The prompt text is parsed into a Settings object. Model and LoRA tags and numeric directives are stripped; style words add descriptor suffixes.

2 The model is chosen: named in the prompt, selected in the Model box, or picked automatically from the style. The model's own step and guidance defaults are applied unless the prompt set them.

3 If the chosen model is not the resident one, the ModelManager frees the current pipeline and loads the new one with its recommended sampler.

4 Requested LoRAs are loaded (once) and activated with their weights; with none requested, adapters are disabled.

5 The prompt and negative prompt are encoded with compel into weighted embeddings (SDXL family) so that (word:1.3) and long prompts work; FLUX receives plain text.

6 For each image: the seed is fixed, peak-memory counters are reset and a telemetry snapshot is taken.

7 The pipeline runs for the requested number of steps at the native size. A per-step callback drives the progress bar.

8 CUDA is synchronised, the inference time is recorded, and a second telemetry snapshot is taken while the GPU is still under load.

9 The native image is saved as PNG. If an upscale target is set, Real-ESRGAN runs and the upscaled PNG is saved.

10 A record with about 50 fields is inserted into generations.db; the gallery, details report, Upscale bar and "Loaded now" line are updated in the browser.

A later click on the Upscale bar reads the native PNG of each selected record, upscales it, saves the result and writes the eight upscale columns onto the same row with db.update().

3.3 Data flow and storage

Data	Where it lives	Format	Written by
Native images	outputs/YYYYMMDD-HHMMSS_seedN.png	PNG, 1 to 2 MB	app.py, generate.py
Upscaled images	outputs/..._seedN_3840px.png	PNG, 8 to 31 MB	app.py
Generation records	generations.db	SQLite, one table	db.py
Built-in model weights	Hugging Face cache	safetensors	downloaded once
User checkpoints and LoRAs	models/checkpoints/, models/loras/	safetensors	the user
Upscaler weights	models/RealESRGAN_x4plus.pth	pth	downloaded once
Logs	stdout of the app process	text	app.py

3.4 Design decisions

Prompt as the only control surface. Early versions had dropdowns for style and size and sliders for steps and guidance. These were removed in favour of a parser because a single text box is faster to use, is self-documenting through the live interpretation line, and makes every generation reproducible from the prompt string alone. The Model box is the one exception, and even it can be overridden from the prompt.

One resident model, swapped on demand. Holding every model in VRAM would need 40 GB. The ModelManager keeps exactly one pipeline, swaps in about 26 s, and verifies the old one is freed. Automatic selection only switches to models that are already on disk so a style word never triggers a surprise 6.5 GB download.

Per-model defaults and samplers. A turbo model wants 6 steps at guidance 2, FLUX wants 4 steps at 0, SDXL wants 30 at 7. The parser records whether the prompt set steps or guidance explicitly; otherwise the model supplies them, and quality words scale them proportionally. Every SDXL-family model runs with the sampler its author recommends instead of the stock Euler scheduler.

compel for prompt weighting. The CLIP encoders truncate at 77 tokens and have no notion of emphasis. compel chunks long prompts and applies (word:1.3) weights to the embeddings. Its own padding helper does not support SDXL's dual encoder, so models.py pads mismatched chunk counts with the embedding of the empty prompt.

Two-stage resolution. SDXL produces incoherent output above roughly 1 megapixel. Generating at native size and upscaling with a super-resolution network is the standard approach and completes in a few seconds. The Upscale bar makes the second stage available after the fact, on the same database row.

Tiled upscaling. Real-ESRGAN on a full 3840 px image would need tens of gigabytes for intermediate feature maps. Processing 512 px tiles with a 32 px overlap bounds memory and hides seams.

Telemetry via nvidia-smi and psutil. Shelling out to nvidia-smi avoids a dependency on NVML bindings and costs about 100 ms per call. psutil covers CPU, memory, disks and network portably, so the Admin panel works unchanged inside the Docker image.

SQLite over a server database. A single-user desktop tool needs no server. The file is trivially backed up and any SQL client can open it. connect() adds missing columns so the schema can grow without migration scripts.

Always-dark theme. The Frazile AI palette is applied to both of Gradio's colour modes so the studio looks the same regardless of the browser's preference.

3.5 Concurrency model

Gradio runs the generate and upscale functions in a worker thread and queues additional requests. The GPU is used by one job at a time (a lock in ModelManager and the "gpu" queue id enforce it, while the statistics handlers run on their own queue). The History and Admin tabs read the database and the system on their own threads; SQLite handles the concurrent reads safely because each call opens its own short-lived connection, and the Admin collectors never touch the pipeline.